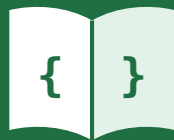

Module 15 — Observability: Metrics, Traces, Logs

Lesson Notes

Spring Boot Edu



2026-09-24

Contents

1. Learning Goals	2
2. Concepts	2
2.1 Three Pillars	2
2.2 Micrometer and the Observation API	2
2.3 Traces and Spans	3
2.4 OTLP and Grafana LGTM	3
3. Step-by-Step Examples	3
3.1 Actuator and Health	4
3.2 Business Metrics with Micrometer	5
3.3 From Micrometer to Prometheus	6
3.4 @Observed	6
3.5 Traces across Services, and Log Correlation	7
3.6 Grafana	8
3.7 Testing Observability	8
4. Common Mistakes and Best Practices	9
5. Summary	9
6. Further Reading	10

1. Learning Goals

By the end of this module you will be able to:

- Explain the three pillars of observability: metrics, traces and logs
- Use Actuator safely: health with custom indicators, probes, metrics, and nothing more than needed
- Record business metrics with Micrometer: counters, timers, gauges
- Observe a method with `@Observed` and get a timer and a span at once
- Follow one request across services with OpenTelemetry tracing
- Write structured JSON logs that carry the trace id
- Send everything to Grafana LGTM and read it on a dashboard

Prerequisites: Module 03 (REST) · **Estimated time:** 5 hours · **Docker required** for Grafana

2. Concepts

2.1 Three Pillars

Signal	Answers	Example	Stored in (LGTm)
Metrics	How much? How fast? How often?	orders per minute, p95 latency	Prometheus/Mimir
Traces	Where did the time of one request go?	HTTP → pricing → catalog	Tempo
Logs	What exactly happened?	“order placed for 9780134685991”	Loki

The signals are most useful when they are **connected**: a latency spike in a metric leads to a slow trace, and the trace id leads to the log lines of that request.

2.2 Micrometer and the Observation API

Micrometer is to metrics what SLF4J is to logging: one API, many backends. Its **Observation API** goes one step further: you describe an operation once (`Observation` or `@Observed`), and handlers turn it into a timer **and** a span. Spring MVC, RestClient, JDBC and many more are observed out of the box.

2.3 Traces and Spans

A **trace** is the whole journey of a request. It consists of **spans**, one per step, each with a start, a duration and a parent. The trace id travels to other services in the W3C `traceparent` HTTP header, so a trace can span many applications.

2.4 OTLP and Grafana LGTM

OTLP (OpenTelemetry Protocol) is the vendor-neutral way to send metrics, traces and logs. Spring Boot 4's `spring-boot-starter-opentelemetry` exports metrics and traces with OTLP. The `grafana/otel-lgtm` image in `compose.yaml` bundles an OpenTelemetry Collector with Loki, Grafana, Tempo and Mimir/Prometheus. It is ideal for development, but not a production setup.

3. Step-by-Step Examples

With Docker running, start the application. Grafana LGTM starts from the root `compose.yaml`, and Boot sends metrics and traces to it automatically:

```
./mvnw -pl modules/15-observability/lesson spring-boot:run
```

Then place a few orders (see `requests.http`) and open Grafana at `http://localhost:3000`.

The configuration:

```
management:
  endpoints:
    web:
      exposure:
        include: health, info, metrics           # never "": heapdump, env, ... leak
  ↪ internals
  endpoint:
    health:
      show-details: always                       # fine locally; in production:
  ↪ when-authorized
      probes:
        enabled: true                           # /actuator/health/liveness and /readiness
  info:
    java:
      enabled: true
  observations:
    annotations:
      enabled: true                             # @Observed
```

```

tracing:
  sampling:
    probability: 1.0           # every request is traced (production:
↪ e.g. 0.1)
metrics:
  tags:
    application: ${spring.application.name} # a common tag on every metric
  otlp:
    metrics:
      export:
        step: 10s             # push metrics every 10 s (default: 1
↪ minute)
logging:
  structured:
    format:
      console: ecs            # one JSON object per line (Elastic
↪ Common Schema)

```

3.1 Actuator and Health

Actuator adds operational endpoints under /actuator. Only health, info and metrics are exposed here. Endpoints such as env or heapdump would reveal configuration values and memory contents.

A custom indicator adds a component to the health status:

```

@Component
class WarehouseHealthIndicator implements HealthIndicator {

    private final Warehouse warehouse;

    WarehouseHealthIndicator(Warehouse warehouse) {
        this.warehouse = warehouse;
    }

    @Override
    public Health health() {
        return warehouse.reachable()
            ? Health.up().withDetail("location", "Istanbul").build()
            : Health.down().withDetail("reason", "no connection to the warehouse
↪ system").build();
    }
}

```

- The component name comes from the bean name: warehouseHealthIndicator → warehouse.

- If any component is DOWN, the whole status is DOWN and the endpoint answers **503**, so a load balancer stops sending traffic to this instance.
- /actuator/health/liveness and /actuator/health/readiness are the groups Kubernetes probes use (module 22).

! Warning

show-details: always is convenient locally. In production use when-authorized: health details can reveal hostnames and versions.

3.2 Business Metrics with Micrometer

```
@Component
public class OrderMetrics {

    private final MeterRegistry registry;
    private final Timer processing;
    private final AtomicInteger inProgress = new AtomicInteger();

    public OrderMetrics(MeterRegistry registry) {
        this.registry = registry;
        this.processing = Timer.builder("bookstore.order.processing")
            .description("Time to place an order")
            .publishPercentiles(0.5, 0.95) // median and p95
            .register(registry);
        Gauge.builder("bookstore.orders.in.progress", inProgress, AtomicInteger::get)
        ↪ // current value, read on demand
            .register(registry);
    }

    public <T> T recordOrder(String channel, Supplier<T> placeOrder) {
        inProgress.incrementAndGet();
        try {
            T result = processing.record(placeOrder);
            Counter.builder("bookstore.orders.placed")
                .tag("channel", channel) // few values
                ↪ only: web, app
                .register(registry) // returns the
                ↪ existing counter
                .increment();
            return result;
        } finally {
            inProgress.decrementAndGet();
        }
    }
}
```

Meter	Measures	Here
Counter	how often something happened (only goes up)	placed orders, per channel
Timer	how long it took, and how often	order processing, with median and p95
Gauge	a current value, read when it is published	orders in progress

Tags split a meter into series, e.g. `channel=web` and `channel=app`. Every distinct tag value creates a new time series, so tags must have **few** possible values. A customer id or an ISBN as a tag can bring down a metrics backend (high cardinality).

```
curl 'localhost:8080/actuator/metrics/bookstore.orders.placed?tag=channel:web'
```

3.3 From Micrometer to Prometheus

Over OTLP, Micrometer's names become Prometheus names with the unit as a suffix. These are the series that arrived in the LGTM stack:

Micrometer	Prometheus
<code>bookstore.orders.placed</code> (counter)	<code>bookstore_orders_placed_total</code>
<code>bookstore.order.processing</code> (timer)	<code>bookstore_order_processing_milliseconds_count</code> , <code>..._sum</code> , <code>...{quantile="0.95"}</code>
<code>bookstore.orders.in.progress</code> (gauge)	<code>bookstore_orders_in_progress</code>
<code>http.server.requests</code> (from Spring MVC)	<code>http_server_requests_milliseconds_count</code> , <code>..._bucket</code> , ...

The common tag application from `management.metrics.tags` is on every series.

3.4 @Observed

```
@Service
public class PricingService {

    @Observed(name = "bookstore.pricing",                // metric name
              contextualName = "calculate-price",        // span name
```

```

        lowCardinalityKeyValues = {"currency", "TRY"})           // tag on both
    public BigDecimal priceOf(String isbn) {
        sleep(50);                                              // pretend to ask a
        ↪ pricing engine
        return isbn.endsWith("1") ? new BigDecimal("89.90") : new BigDecimal("55.00");
    }

```

One annotation gives a timer `bookstore.pricing` (with the tag `currency=TRY`) and a span `calculate-price` in every trace that passes through this method. It needs `spring-boot-starter-aspectj` and `management.observations.annotations.enabled=true`.

3.5 Traces across Services, and Log Correlation

The order controller calls the “catalog service” over HTTP. Because the `RestClient` is built from Boot’s `RestClient.Builder`, the call is observed and the trace context is sent along:

```

@Component
public class CatalogClient {

    private final RestClient.Builder http;
    private final Environment environment;

    public CatalogClient(RestClient.Builder http, Environment environment) { // the
        ↪ builder, not RestClient.create()
        this.http = http;
        this.environment = environment;
    }

    public String titleOf(String isbn) {
        String port = environment.getProperty("local.server.port", "8080");
        Map<?, ?> book = http.baseUrl("http://localhost:" + port).build()
            .get().uri("/api/catalog/{isbn}", isbn)
            .retrieve().body(Map.class);
        return String.valueOf(Objects.requireNonNull(book).get("title"));
    }
}

```

One order therefore produces one trace:

```

POST /api/orders                (server span)
├─ calculate-price              (@Observed, ≈ 50 ms)
├─ GET /api/catalog/{isbn}      (client span)
└─ GET /api/catalog/{isbn}      (server span of the "other service")

```

With `logging.structured.format.console: ecs`, every log line is a JSON object. During a request, it also carries the trace and span id:

```
{"@timestamp":"2026-09-
↪ 24T19:27:17.777Z","log":{"level":"INFO","logger":"...OrderController"},
"message":"order placed for 9780134685991 via web",
"traceId":"349b7da28c9e1f1eff65fdb9c95f36644","spanId":"14d5b6755a51c2c5", ...}
```

The log lines of the controller and of the catalog carry the **same** `traceId`. `TraceCorrelationTest` checks exactly this. Search your logs for a trace id from Grafana, and you find everything that happened in that request.

i Note

Boot 4 prepares an OTLP log exporter too, but log events reach it only through an OpenTelemetry Logback appender. That appender is not managed by Spring Boot's BOM and is not used in this course. Here the logs go to the console as JSON. In production, a log collector (e.g. the OpenTelemetry Collector or Grafana Alloy) ships them to Loki.

3.6 Grafana

In Grafana (<http://localhost:3000>):

- **Explore → Tempo**: search the traces of 15-observability, and open one to see the spans above as a timeline.
- **Explore → Prometheus**: e.g. `sum by (channel) (rate(bookstore_orders_placed_total[1m])) * 60`.

The module contains a ready-made dashboard with five panels (orders per minute by channel, processing time, pricing time, HTTP requests, orders in progress). Import it once:

```
curl -X POST -H 'Content-Type: application/json' \
  -d @modules/15-observability/grafana/bookstore-orders-dashboard.json \
  http://localhost:3000/api/dashboards/db
```

3.7 Testing Observability

- In tests, Boot keeps observations but **does not export** metrics or traces. `@AutoConfigureTracing` gives a real tracer for assertions about trace ids.
- `TestObservationRegistry` (from `micrometer-observation-test`) checks observations and their parents without any backend (exercise 2).
- `GrafanaLgtmIT` tests the whole chain: a Testcontainers LGTM stack, export switched back on, and assertions against Prometheus and Tempo:

```
@TestConfiguration(proxyBeanMethods = false)
static class Lgtm {

    static final LgtmStackContainer LGTM = new
        ↪ LgtmStackContainer("grafana/otel-lgtm:0.33.1"); // as compose.yaml

    @Bean
    @ServiceConnection // OTLP endpoints for metrics,
        ↪ traces and logs
    LgtmStackContainer lgtm() {
        return LGTM;
    }
}
```

4. Common Mistakes and Best Practices

!! Caution

Never use unbounded values (user ids, ISBNs, URLs with ids) as metric tags. Each value creates a new time series, and the metrics backend runs out of memory. Put such values into traces or logs instead.

- **Do:** expose only the Actuator endpoints you need, and protect them (separate port, Spring Security).
- **Don't:** sample 100 % of the traces in a busy production system. Start with a small `sampling.probability` and raise it where needed.
- **Do:** name metrics after what they measure, with dots (`bookstore.orders.placed`). Micrometer converts the names for each backend.
- **Don't:** log a message and also count it by parsing logs. Count with a metric, explain with a log line.
- **Do:** make logs structured (JSON) and include the trace id, so that logs, traces and metrics can be connected.
- **Don't:** let a health indicator call slow or flaky systems without a timeout. A slow health check makes the whole instance look unhealthy.

5. Summary

- Metrics say how much, traces say where, logs say what. Connected by the trace id, they explain a production problem.
- Actuator gives health (with custom indicators and probes) and metrics. Expose as little as possible.
- Micrometer records counters, timers and gauges. Tags must have few values.

- `@Observed` and Spring's built-in observations produce timers and spans together.
- `RestClient.Builder` carries the trace to the next service. ECS JSON logs contain `traceId` and `spanId`.
- `spring-boot-starter-opentelemetry` sends metrics and traces over OTLP to Grafana LGTM, where Prometheus and Tempo show them.

Next step: [Exercises](#)

6. Further Reading

- [Spring Boot — Actuator · Metrics · Tracing · Observability](#)
- [Spring Boot — Structured Logging](#)
- [Micrometer Documentation · Observation API](#)
- [OpenTelemetry — Concepts](#)
- [Grafana docker-otel-lgtm](#)